

Glossário de Termos — Aula 01

Introdução à Orientação a Objetos

Este glossário reúne os termos fundamentais da Aula 01 — o ponto de partida da Orientação a Objetos em Java. Você vai sair daqui sabendo o que é uma classe, como criar objetos, controlar o estado interno e proteger seus dados com encapsulamento. Use como referência durante os exercícios.

Pilar 1 — Classe e Objeto

Classe (Molde, planta, definição)

Estrutura que descreve as características (atributos) e os comportamentos (métodos) que todos os objetos daquele tipo terão. A classe em si não ocupa espaço na memória — ela é só o projeto.

Analogia: A planta de uma casa é uma classe. Ela descreve quantos cômodos, paredes e portas a casa terá, mas você não mora na planta — mora numa casa construída a partir dela.

```
public class Carro {  
    String modelo;    // atributo  
    int    ano;        // atributo  
    void    ligar() { ... }    // método (comportamento)  
}
```

Dica: Por convenção em Java, nomes de classe começam com letra MAIÚSCULA e seguem o padrão PascalCase (ex: ContaBancaria, ClienteVip).

Objeto (Instância concreta de uma classe)

Cópia 'viva' da classe na memória, com valores próprios para cada atributo. Cada objeto tem identidade própria, mesmo que dois objetos tenham os mesmos valores.

Analogia: Se a classe Carro é a planta, o objeto é o Fusca azul de placa ABC-1234 estacionado na rua. Existem milhões de carros no mundo — todos seguem alguma 'planta', mas cada um é único.

```
Carro c1 = new Carro();  
Carro c2 = new Carro();    // c1 e c2 sao objetos diferentes,  
                           // mesmo que tenham atributos iguais.
```

Dica: Comparar objetos com == verifica se as duas variáveis apontam para a MESMA posição de memória — não compara conteúdo. Para isso existe equals().

Instanciação (Ato de criar um objeto a partir da classe)

Processo de pedir ao Java para reservar memória e criar um objeto novo a partir de uma classe. A palavra-chave new é o disparo da instanciação.

```
Carro meuCarro = new Carro();  
// Tipo declarado | nome | new | classe | construtor
```

Dica: Sem new, não existe objeto. Tentar acessar atributos ou métodos antes de instanciar resulta em NullPointerException em tempo de execução.

Pilar 2 — Membros da classe

Atributo (Característica/dado armazenado no objeto)

Variável declarada DENTRO da classe (fora de métodos). Cada objeto possui sua própria cópia de cada atributo, com valor independente dos demais objetos.

```
public class Pessoa {  
    String nome;    // atributo  
    int idade;     // atributo  
}
```

Dica: Atributos não inicializados recebem valor padrão: 0 para números, false para boolean, null para objetos e String.

Método (Ação/comportamento do objeto)

Bloco de código declarado dentro da classe que descreve uma operação. Métodos podem ler/alterar atributos do próprio objeto, receber parâmetros e devolver valores. São acessados via operador ponto (objeto.metodo()).

```
public class Lampada {  
    boolean ligada;  
    public void ligar() {  
        ligada = true;  
    }  
    public boolean estaLigada() {  
        return ligada;  
    }  
}
```

Dica: Por convenção, nomes de métodos começam com letra minúscula e seguem camelCase (ex: calcularTotal, getNome). Verbos costumam ser bons nomes.

Estado do objeto (Conjunto atual dos valores dos atributos)

Em qualquer instante, o estado de um objeto é a 'foto' dos valores que seus atributos possuem naquele momento. Métodos podem ler e alterar esse estado.

Analogia: O estado de uma lâmpada pode ser: ligada=true, cor='amarela', potencia=60. Apertar o interruptor é um método que muda o estado para ligada=false.

```
Lampada sala = new Lampada();  
sala.ligar();           // muda o estado  
System.out.println(sala.estaLigada()); // true
```

Pilar 3 — Construtor

Construtor (Método especial de inicialização)

Método invocado automaticamente quando o objeto é criado com `new`. Tem o mesmo nome da classe e nunca declara tipo de retorno. Serve para inicializar atributos com valores coerentes desde o nascimento do objeto.

Analogia: É como a ficha de cadastro preenchida no momento em que um aluno entra na escola — sem ela, o sistema teria um aluno sem nome, sem matrícula, sem informação alguma.

```
public class Carro {
    String modelo;
    int ano;
    public Carro(String modelo, int ano) {
        this.modelo = modelo;
        this.ano = ano;
    }
}
// Uso:
Carro c = new Carro("Fusca", 1975);
```

Dica: Se você não escrever nenhum construtor, o Java cria automaticamente um construtor padrão sem parâmetros. Mas se você definir ao menos um construtor personalizado, esse padrão deixa de existir — então cuidado!

this (Referência ao próprio objeto)

Palavra-chave usada dentro de métodos e construtores para se referir ao objeto atual. Útil sobretudo quando o nome de um parâmetro coincide com o nome de um atributo — `this.modelo` é o atributo, `modelo` é o parâmetro.

```
public Carro(String modelo) {
    this.modelo = modelo; // atributo = parametro
}
```

Pilar 4 — Encapsulamento

Encapsulamento (Proteção do estado interno do objeto)

Princípio que recomenda esconder os atributos da classe (declarando-os como `private`) e expor o acesso a eles apenas por métodos controlados (getters e setters). Garante que ninguém possa alterar o estado de forma inválida.

Analogia: É como o vidro do extintor de incêndio: você consegue ver e usar quando precisa, mas para mexer no conteúdo precisa quebrar o vidro — e isso é uma ação controlada, não algo que qualquer pessoa faz à toa.

private / public (Modificadores de acesso)

`private` restringe o acesso apenas para dentro da própria classe. `public` libera para qualquer parte do programa. O encapsulamento típico marca atributos como `private` e métodos como `public`.

```
public class Conta {
    private double saldo; // ninguém mexe direto
    public double getSaldo() { // todos podem consultar
        return saldo;
    }
}
```

Dica: Existem ainda os modificadores `protected` (acesso por subclasses) e o nível de pacote (sem modificador), que serão aprofundados nas aulas seguintes.

Pilar 5 — Getters e Setters

Getter (Método de leitura de um atributo)

Método público que devolve o valor de um atributo privado. Por convenção, o nome começa com get seguido do nome do atributo em PascalCase.

```
private String nome;

public String getNome() {
    return nome;
}
```

Dica: Para atributos do tipo boolean, a convenção troca get por is (isAtivo, isEstudante). Soa mais natural quando lido em voz alta.

Setter (Método de escrita controlada de um atributo)

Método público que recebe um valor e o atribui ao atributo privado. É o lugar perfeito para validar o valor antes de aceitar — protegendo o estado do objeto contra dados inválidos.

```
private int idade;

public void setIdade(int idade) {
    if (idade < 0) {
        throw new IllegalArgumentException("Idade nao pode ser negativa.");
    }
    this.idade = idade;
}
```

Dica: Nem todo atributo precisa de setter. Se um valor não deve mudar depois de criado (ex: CPF, número da conta), basta o getter — e marque o atributo como final para reforçar a imutabilidade.

Outros conceitos da aula

Referência (Variável que aponta para um objeto)

Em Java, variáveis de tipo classe não guardam o objeto em si — guardam um endereço para ele. Por isso atribuir um objeto a duas variáveis NÃO copia o objeto; faz com que ambas apontem para o mesmo dado.

```
Carro a = new Carro();
Carro b = a;           // a e b apontam para o MESMO carro
b.modelo = "Gol";
System.out.println(a.modelo); // "Gol" — mexeu nos dois
```

null (Ausência de objeto)

Valor especial que indica que uma variável de tipo classe não aponta para objeto algum. Tentar chamar um método de uma variável null lança NullPointerException em tempo de execução.

new (Operador de instanciação)

Operador que aloca memória no heap para um novo objeto, invoca o construtor e devolve a referência. Sem new, não há objeto.